`GardenBench`: A lightweight, daily evaluation of LLM capabilities*

Benedict Cummins-Mburu Mariana Garcia Mejia Oscar Heath
Sabrina Kreyzerman Ellie Murray Arusan Surendiran
Rohan Alexander

July 30, 2026

We introduce `GardenBench`, a benchmark with 11 test suites that evaluate the capabilities of Large Language Models (LLMs) across a range of tasks. By systematically prompting LLMs with various lightweight queries, we document day-to-day changes in model performance, including run costs and times, that other benchmarks miss. This allows us to compare the accuracy and features of responses across model providers and versions. We account for daily variations in model accuracy, capturing both model progress and degradation. Our benchmarking pipeline is reliable and scalable as we evaluate accuracy as a binary variable (correct or incorrect) using exact-match strings on deterministic ground truths. We base our evaluation on general-use tasks and make our results publicly accessible via a live dashboard, diversifying existing approaches to LLM testing. Our main contribution is an innovative, lightweight, and inexpensive benchmark, designed to be consumer-facing and for everyday tasks.

Keywords: Artificial intelligence; AI; Benchmarks; Large Language Models; LLMs

*Code and data are available at: https://github.com/RohanAlexander/daily_benchmark. The live dashboard is available at: https://gardenbench.ai/. We gratefully acknowledge funding from Natural Sciences and Engineering Research Council of Canada (NSERC) Alliance Grant (110_2024_2025_Q4_13) and Social Sciences and Humanities Research Council (SSHRC) Partnership Grant (895-2025-1002). This research was conducted while Benedict Cummins-Mburu, Mariana Garcia Mejia, Oscar Heath, Sabrina Kreyzerman, Ellie Murray and Arusan Surendiran were interns at the Investigative Journalism Foundation. We thank Zane Schwartz for comments and suggestions. Correspondence: rohan.alexander@utoronto.ca.

One must cultivate one's own garden.
—Voltaire

1 Introduction

Progress depends on the instruments we use to measure it. Traditionally, machine learning (ML) models were trained for and tested on task-specific functions. For example, models were deliberately trained to perform visual recognition and object classification in the ImageNet challenge (Russakovsky et al. 2015) and movie recommendation enhancement in the Netflix Prize contest (Bell et al. 2010). But in 2020, the release of GPT-3 (Brown et al. 2020), one of the first publicly accessible and general-use large transformer models, shifted the trajectory of LLM training and testing. Brown et al. (2020) found that sufficiently large transformer models that were trained only on next-token prediction could perform competitively with fine-tuned state-of-the-art (SOTA) models on tasks that were not in their training data. This property of LLMs drove their rapid spread as general-purpose tools. Today, LLMs are used as search engines, office assistants, and personal tutors, among other general use cases. LLMs are referred to as foundation models (Bommasani et al. 2022; Liang et al. 2023), used to perform a broad range of tasks, including text summarization, translation, coding, image generation, and answering practical questions. LLMs are also often able to process multimodal inputs, such as image files and PDF documents.

Though LLM use has broadened, the evaluation and documentation of LLM performance remains scientifically oriented and domain-specific. In addition, many model benchmarks are computationally expensive and take long to run (Polo et al. 2024). As a result, model providers assess overall performance by evaluating newly released models once upon their release. Model capabilities across specialized tasks are reported in model cards, for example, GPT-5.5 (OpenAI 2026a), Gemini 3.5 Flash (Google DeepMind 2026), and Claude Opus 5 (Anthropic 2026b). We identify two gaps in this practice. First, model cards are published by model developers, allowing them to selectively showcase model abilities and exclude failures. The same applies for many benchmarks, with model developers either choosing which benchmark performances to publish or constructing their own (for example, Anthropic's `BioMysteryBench` to evaluate Claude Fable 5). Second, early one-time evaluation of models does not capture changes in performance, whether model progress or model drift.

In this paper, we respond to calls for the adoption of publicly accessible benchmarks (for example, Guha et al. 2026), as well as benchmarks that evaluate model performance more frequently and on diverse and general use cases (for example, Hodak et al. 2024; Liang et al. 2023). This kind of wide-ranging evaluation is intended to align LLM benchmarks with broader LLM use.

We introduce `GardenBench`, a lightweight, fast, and inexpensive benchmark designed to run daily on all frontier models. `GardenBench` is an LLM benchmark with 105 tests organized into 11 suites with varying tasks and difficulty levels. We use Internet-sourced images, author-collected photos, and standardized text prompts to evaluate model performance against exact-match strings on deterministic ground truths. We test frontier models from Anthropic, Deepseek, OpenAI, Google, Moonshot AI, Alibaba Cloud, and Cohere by running our benchmark once per day. This process was automated using the GitHub Actions workflow outlined in De Angelis (2026).

We expand on existing benchmarks in a number of ways. Primarily, our tests comprise a combination of innovative and practical tasks. Whereas many benchmarks focus on specific skills, such as coding and image generation, our tests measure model capability across tasks that require multiple skills, such as image recognition, logical and arithmetic reasoning, and information extraction. Second, the complete benchmark costs less than $50 USD in total to run, allowing for personal use by interested parties. Third, our tests run every day, with the results systematically published to a publicly accessible live dashboard[1].

The remainder of this paper is structured as follows. Section 2 provides a contextual background, where we identify and address shortcomings in existing benchmarks. Section 3 details our experimental approach, including the construction of our dataset and the models that are evaluated. Section 4 covers our findings regarding model performance, evaluated against metrics such as run cost and time. Section 5 discusses key takeaways, implications, weaknesses, and future directions.

2 Background

In ML, benchmarks are collections of questions and answers that assess LLM performance on different domains of knowledge, such as math and biology, as well as tasks, such as text summarizing and coding. Traditionally, benchmarks are designed to test specific capabilities and compare models through objective metrics (Raji et al. 2021). They are widely used to quantitatively track the progress of LLMs and rank models by performance, as well as to identify the most promising technologies to guide research efforts (Zhu et al. 2025).

`GardenBench` addresses two caveats identified in the existing work on LLM benchmarking. First, LLM benchmarks tend to be highly specialized, making results less useful to the broader public and dominated by field professionals (Koch et al. 2021). Second, evaluation by general-use benchmarks is often costly, meaning these tests are typically only run once upon the release of a new model.

[1] https://gardenbench.ai/

2.1 Transparency concerns in domain-specific benchmarks

Concerns about transparency in existing LLM benchmarks are well documented and characterized by several key shortcomings. Namely, scholars warn that LLM evaluation occurs predominantly as a closed-door practice, where model providers selectively publish their own evaluations (Cheng et al. 2025; Baack et al. 2026), benchmark results target niche professional communities (Koch et al. 2021), and evaluation metrics are not useful to the broad scope of LLM users (Raji et al. 2021; Eriksson et al. 2025). For example, Guha et al. (2026) calls for more publicly accessible LLM benchmarks, with other scholars questioning the trustworthiness of current LLM benchmarking practices (Eriksson et al. 2025; Cheng et al. 2025).

Conventionally, newly released ML models are evaluated on a set of highly specialized benchmarks, with the results used to demonstrate the model's strengths. This practice enables providers to publish benchmarks on which their models perform well, while less successful performances remain undisclosed (Cheng et al. 2025; Baack et al. 2026). Bommasani et al. (2023) introduced the Foundation Model Transparency Index (FMTI) in 2023 to expose and improve the disclosure practices of frontier model developers, such as Anthropic, Google, and OpenAI (for the most recent version, see Wan et al. (2025)). The FMTI is an effort to advance the transparency of model developers across their computational methods, labor practices, the real-world implications of LLMs, among other metrics.

Moreover, it has become increasingly difficult for the general public to interpret benchmark results as they are often highly technical and domain-specific. Narayanan (2026) finds that there is a discrepancy between LLM potential and real LLM applications. They suggest one reason may be that the LLM capabilities demonstrated by model developers and real-world applications are misaligned. This position is also found in Eriksson et al. (2025), Hutchinson et al. (2022), and Cheng et al. (2025). In addition, we suspect this may also be because current approaches to LLM evaluation may not measure what is relevant or useful to the broader community of LLM users. For example, when Anthropic released Claude Fable 5 in July 2026, the company highlighted results from `SWE-bench` (Jimenez et al. 2024), `TerminalBench` (Reuel et al. 2024), `ExploitBench` (Lee and Brumley 2026), and `BioMysteryBench` (Anthropic 2026a). Additionally, `BioMysteryBench` was constructed by Anthropic to test its own model. Respectively, these benchmarks test agentic coding, command-line coding, cybersecurity, and biology. GPT-5.6 Sol was also evaluated on similar measures (see Reuel et al. 2024; Li and Ho 2026; Lee and Brumley 2026; Wang et al. 2026). While these insights are important for model development and medical applications, these metrics may not be useful to LLM users outside professional communities, who use LLMs for ordinary, everyday tasks. This feature of existing LLM testing, dominated by the providers themselves, supports the need for general-purpose benchmarks that are publicly accessible, conceivable, and that indiscriminately test a wide range of capabilities. We respond by developing a benchmark that tests general contextual knowledge, visual perception and object identification, and simple logical and arithmetic reasoning. We also publicly upload our results daily to a live dashboard.

2.2 Expensive and infrequent model testing

The expensive cost of many LLM benchmarks is a second vulnerability of existing approaches. Gundlach et al. (2026) found that the cost of benchmarking frontier models is increasing by 3 to 18 times yearly, as models continue to grow in size and require more internal thinking before outputting responses. For example, Kapoor et al. (2025)'s "Holistic Agentic Leaderboard" costs $40,000 USD to run once. Gundlach et al. (2026) call for more attention to the computational resources needed to evaluate LLM performance, taking realistic economic constraints into account when assessing LLM applications.

Infrequent testing is one consequence of the expensive cost of LLM evaluation. Mitchell et al. (2019) proposed model cards to accompany newly released models as a report of the model's strongest capability domains. While model cards offer a step forward in transparently documenting model performance, the tests performed are often treated as a one-time evaluation of the model. Example model cards from the models included in our study are GPT-5.5 (OpenAI 2026a), Gemini 3.5 Flash (Google DeepMind 2026), and Claude Opus 5 (Anthropic 2026b). While model cards comprehensively evaluate model performance across various capabilities, they do not account for model, or temporal drift—the performance degradation of LLMs as data and language patterns, as well as domain-specific knowledge, change over time (Khairnar 2025). As a result, model cards quickly become outdated as new model versions are released. For example, Chen et al. (2024) document how the performance of GPT-3.5 and GPT 4, for instance in math problems, degraded in a matter of months. Additionally, model cards are published by model developers, reinforcing doubts about transparency and impartiality.

2.3 A new notion of reliability

Questions about the reliability of current LLM benchmarks are raised by both providers' selective disclosure of model development and performance, as well as their expensive and infrequent qualities.

Reliability is a critical feature of LLM evaluation, with many studies examining the consistency and reproducibility of LLM outputs. For example, Cui and Alexander (2026) examine the ability of LLMs to produce the same response to the same prompt ten-out-of-ten times. Many studies take a similar approach (for instance, Rabanser et al. 2026; Pinhanez et al. 2025; Ahn and Yin 2025; Patwardhan et al. 2025).

Rabanser et al. (2026) conceptualize reliability by four dimensions (consistency, robustness, predictability, and safety). With `GardenBench`, we introduce a new notion of reliability by testing whether the same prompt asked to the same model at the same time, on different days, produces the same response. We argue that the consistency of responses across days of the week is equally an important component of reliability. However, the expensive cost of running LLM benchmarks does not allow for this kind of analysis, which requires frequent recurring testing. Accordingly, `GardenBench` costs less than $50 USD to run the complete composition of tests, enabling us to run the benchmark daily. This uniquely allows us to evaluate LLM responses to the same prompt each day of the week, document day-to-day consistency, and capture model drift.

2.4 Related work

Arena, formerly ChatBot Arena (Chiang et al. 2024), and EpochAI (Epoch AI 2026) are examples of public dashboards that report LLM capabilities across a variety of skills, models, and benchmarks. We follow these dashboards in adopting independent and public-facing testing of LLMs, while addressing areas for improvement. First, Arena is a crowd-sourced approach that allows public users to rank their preferred models on a variety of tasks. This enables frequent evaluation of new models, but relies on the participation of the public, meaning the dashboard does not update if people are not using it. Additionally, Arena does not allow for quantitative comparisons between models. Similarly, EpochAI primarily examines model accuracy against its release date without accounting for potential trade-offs when seeking high performance, such as the time and cost of testing. The models also were not tested beyond their initial release.

`GardenBench` was designed with consideration to the shortcomings of existing approaches to LLM benchmarking. We propose a diversified benchmark that includes a wide variety of everyday tasks that reflect general use cases, many of which can be accomplished by humans. We test LLM capabilities ranging from contextual knowledge to image recognition in a way that is lightweight, cheap, and fast to run. In doing so, we contribute to academic efforts in the movement toward publicly accessible and general-use benchmarks.

3 Data and methods

3.1 Developing testing suites

In constructing our testing methodology, we follow extensive literature on prompt engineering (for example, Ggaliwango et al. 2024; Sahoo et al. 2025) and the instruction-following capabilities of LLMs (for example, Zhou et al. 2023; Wu et al. 2024). These studies find that developing specific and intentional query structures, combined with length and formatting constraints (for instance, capitalization and punctuation), is critical in maximizing the utility and efficacy of LLMs.

Our benchmark consists of 105 individual tests organized into 11 distinct testing suites. A brief overview of each suite and their associated tasks is available in Table 1. We distinguish between image- and text-based suites because some of the models we evaluated, such as DeepSeek, are not natively capable of image processing. Similarly, sound-based tasks were initially considered (for example, recognizing songs from audio recordings), but ultimately discarded as few models in 2026 natively support audio file processing[2]. As we evaluate model capabilities without tool use, audio-based tasks were not feasible. For our final collection of tasks, all correct answers could be found within the image or text in the prompt, meaning web searching was not required. Our objective in this approach was to evaluate raw LLM capabilities without the support of external resources.

Tests in the same suite either evaluate LLMs on different versions of the same task or multiple closely related but distinct tasks. For instance, the *Birds* suite consists of 10 instances of the same task: "Identify the species of the bird shown in this image." Conversely, the *Books* suite contains a single instance of 10 different tasks, all of which ask the LLM to extract different information from a photograph of a bookshelf (for example, the title and author of one of the books).

Our collection of suites reflects diverse everyday tasks that test a broad range of LLM capabilities while being effectively cost-efficient to run. The *Ciphers*, *Puzzles*, *Clothing*, *Transit*, and *Colours* suites test the model's ability to reason across different domains and based on different media input types. The *Countries* and *Birds* suites test base LLM knowledge, when access to tools such as web searching was restricted. The *Haystack* and *Books* suites test information retrieval. We also included a self-check, where the LLM was prompted to return its own model slug, a string used to identify a specific model.

We only considered tasks which could be evaluated on a binary pass or fail. This was done to make the results of `GardenBench` easily interpretable. In addition, models were only given a single chance to respond, and example responses were not provided in the prompts. This zero-shot prompting style is intended to emulate how a human with a quick question would interact with an LLM. Model responses were evaluated using exact string matching between the model output and the test's ground truth. To support reliability and scalability, the exact same bytes were sent to each API. Moreover, prompts each demanded a specific response structure from the LLM with no pleasantries, throat clearing, or other excess communication. Therefore, our approach simplifies evaluation, making results easily conceivable to broad LLM users without sacrificing depth of analysis. An example prompt from the *Ciphers* suite is shown in Figure 1. A more complete description of our suites is available in Appendix A.

3.2 Models

We evaluated the performance on our benchmark of 17 different models developed by 7 distinct providers, shown in Table 2. Models were prompted in Python using the OpenAI (2026b) package. The exact API string of each model is available in Appendix B. This set of models was chosen for its diversity, encompassing open- and closed-weight models developed in the U.S., Canada, and China. These models also differ in their novelty and size. For example, Claude Haiku 4.5 was released in October 2025 and has 20 billion parameters. Comparatively,

[2]Many large providers offer specialized models for audio-based tasks such as speech-to-text processing (e.g. OpenAI's Whisper).

Table 1: Full list of suites considered in our study with their reference names, number of tests they contain, and a one-sentence task description.

Suite Name	Number of Tests	Input Type	Task Description
Birds	10	Image	Identify the species of bird from its image
Books	9	Image	Identify information from an image of a bookshelf.
Clothes	10	Image	Identify information from an image of a wardrobe.
Colours	12	Image	Identify paint types and colour mixtures from an image of a paint palette.
Countries	11	Image	Identify the country from an image of its silhouette.
Graphs	10	Image	Identify the equation of the derivative from a graph of a simple function.
Transit	10	Image	Identify information from a map of a transit system.
Ciphers	10	Text	Decode a word transformed with a cipher.
Haystack	8	Text	Identify a target string within a large corpus of text.
Puzzles	12	Text	Solve logical puzzles.
Self-Check	5	Text	Answer questions about self.

Prompt:

```
Task: Decode the hidden word using the given transformation rule.
Alphabet mapping: A=0, B=1, ..., Z=25
Rule: f(x)=25-x, where x represents the integer value of each letter.
Output Constraint: Return only the decoded word. Do not include any
explanation, preamble, or punctuation.
Encoded Word: HOZBVW
```

Example LLM Response:

```
SLAYED.
```

Figure 1: Prompt used for a single instance of the task tested by the *Ciphers* suite, with an example LLM response. The response shown is evaluated as "Passed" since it matches the associated ground truth.

GPT 5.6 Sol was released in July 2026 and is estimated to have 2 to 4 trillion parameters. This allows us to compare model performance across a variety of providers and versions as they develop over time. Lastly, these models are popularly used among the general public, making their evaluation useful to broad communities.

The set of models presented in this paper will change as providers release new model versions and discontinue older ones. As a result, `GardenBench` provides a permanent timeline of a model's performance on everyday tasks. This means that changes in performance, for instance, model drift, will be captured in the timeline of our benchmark.

3.3 Evaluation schedule

All models were evaluated daily We began systematically running the benchmark on August 01, 2026, with runs scheduled at 4:00AM Eastern Standard Time (EST) each day. A cumulative dataset of historic runs was automatically updated using the GitHub Actions workflow detailed in De Angelis (2026). Figure 2 presents a high-level overview of our research workflow.

Our primary product is a dataset of daily `GardenBench` evaluations, which are updated and made publicly available on our website[3]. To preserve the integrity of our experiment while running `GardenBench` daily, we have chosen not to make our underlying dataset public. This decision was informed by concerns about dataset contamination, where model training data and evaluation data may intersect, enabling LLMs to memorize responses, thus causing performance to be overstated (Xu et al. 2024; Fu et al. 2025; Li et al. 2024). There is also less literature on how to navigate the risk of data contamination for emerging dynamic benchmarks (Chen et al. 2025), such as ours.

[3]https://gardenbench.ai/

Table 2: Full list of models evaluated in our study, organized by their providers, with their names, native image support capability, and frequency of evaluation.

Provider	Model Name	Image Support	Evaluation Frequency
Alibaba Cloud (via Fireworks)	Qwen 3.7 Plus	Yes	Daily
Anthropic	Fable 5	Yes	Bi-weekly
Anthropic	Sonnet 4.6	Yes	Daily
Anthropic	Haiku 4.5	Yes	Daily
Anthropic	Opus 4.8	Yes	Weekly
Anthropic	Opus 5	Yes	Weekly
Cohere	Command-A+	Yes	Daily
Cohere	Command-R+	Yes	Daily
Deepseek	V4 Pro	No	Daily
Deepseek	V4 Flash	No	Daily
Google	Gemini 3.5 flash	Yes	Daily
Google	Gemini 3.1 pro	Yes	Weekly
Moonshot AI (via Fireworks)	Kimi K2.6	Yes	Daily
Moonshot AI (via Fireworks)	Kimi K3	Yes	Daily
OpenAI	GPT 5.6 Sol	Yes	Bi-weekly
OpenAI	GPT 5.4 mini	Yes	Daily
OpenAI	GPT 5.5	Yes	Weekly

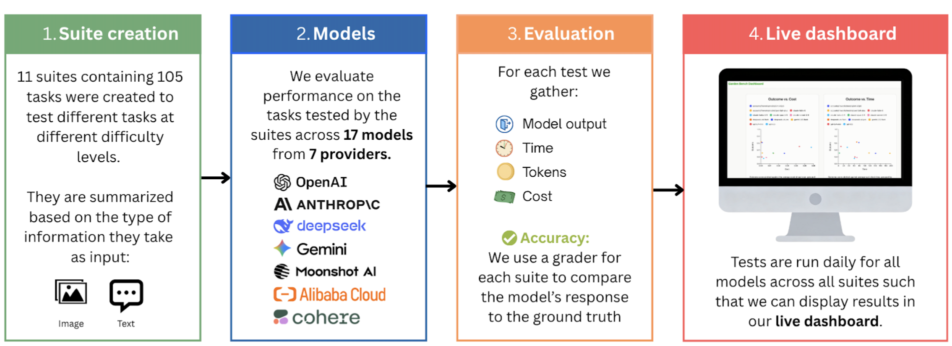

Figure 2: Research workflow.

4 Results

4.1 Aggregate performance

Figure 3 shows the accuracy of each model on the complete benchmark. In other words, the average accuracy across all our tests from all 11 suites for each model. The highest performing models were DeepSeek V4 Flash and DeepSeek V4 Pro, achieving an average accuracy rate of approximately 71.2 percent and 67.7 percent, respectively. Importantly, we note that these models (Deepseek V4 Pro and Deepseek V4 Flash) were evaluated only on text-based suites as they did not natively support image inputs. Only 4 test suites were composed solely of text prompts, compared to 7 that included images, which could have influenced the model's accuracy calculated here, relative to models that were tested on all 11 suites.

Apart from DeepSeek, we observe notable variation in the average performance of model versions from the same model providers. For example, GPT-5.4 Mini was among the lowest performing models (31.3 percent average accuracy), while GPT-5.6 Sol and GPT-5.5 ranked directly behind DeepSeek models in performance, at 67.7 percent and 65.6 percent accuracy, respectively. Similarly, the performance of Anthropic models steadily increased across versions — Claude Fable 5 performed most successfully across Anthropic models (51.3 percent), followed by Claude Opus 4.8 (36.4 percent), Claude Sonnet 4.6 (31.5 percent), and Claude Haiku 4.5 (23.0 percent). These patterns support that newer model versions exhibit improved performance.

Fireworks-hosted models Kimi K3, Qwen 3+, and Kimi K2.6 performed with 61.9 percent, 56.9 percent, and 48.9 percent average accuracy, respectively. Cohere's Command A+ followed in performance with 36.9 percent average accuracy. Lastly, Command R+ was the lowest performing model at 16.8 percent average accuracy. Command R+ also did not natively support image inputs, and therefore, was only evaluated on 4 text-based suites. This limitation is important because it means these Deepseek models and Command-R+ are not directly comparable with models that were tested on all 11 suites.

4.2 Overall accuracy per suite

Next, we examine average accuracy across all models per suite. Figure 4 shows the average accuracy across all models in each testing suite, allowing us to interpret the kind of tasks in which LLMs are most successful and identify the tasks they struggled with. Model performance was highest in the *Birds* suite (85 percent), where LLMs were asked to identify the species of bird from an Internet-sourced image.

Models performed similarly across the remainder of text- and image-based suites, where average accuracy ranged from around 50 to 70 percent across 6 suites (*Haystack*, *Countries*, *Graphs*, *Colours*, *Ciphers*, and *Puzzles*). This small variation in model performance across text- and image-based suites suggests that models that support both input types performed similarly whether or not prompts included image attachments. Specifically, the incorporation of image

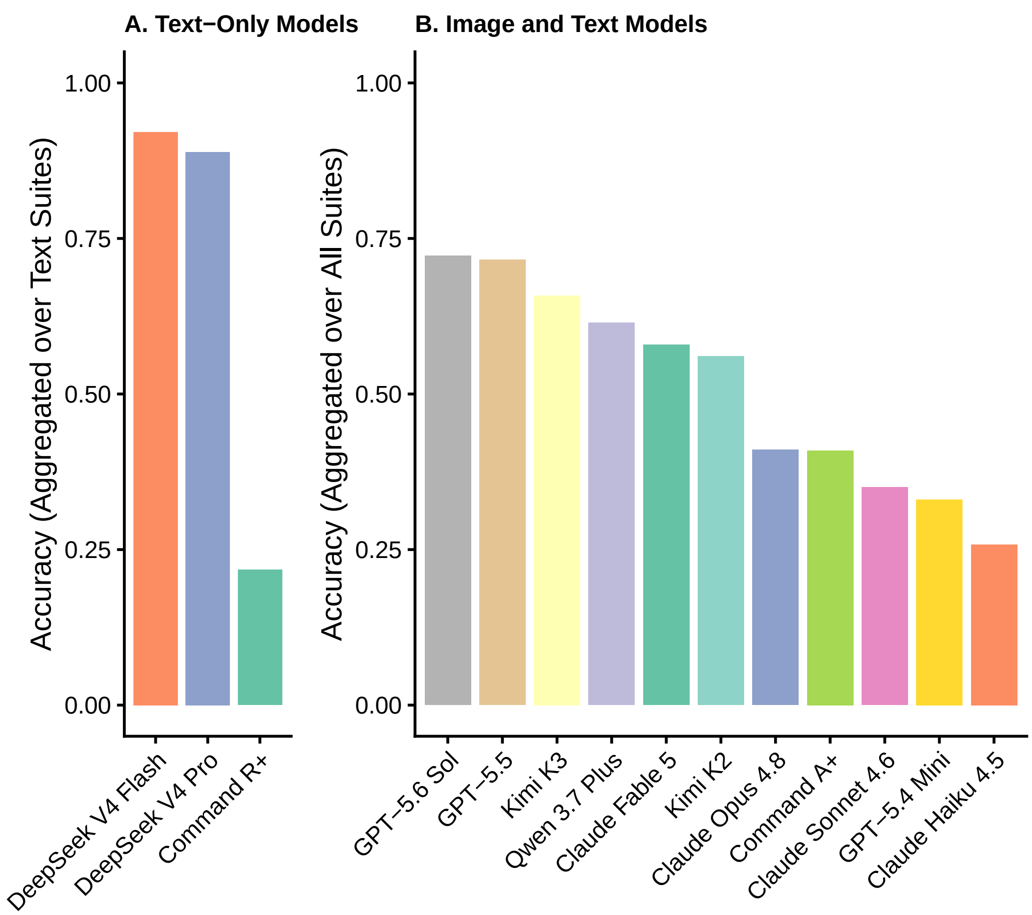

Figure 3: Performance by model, separated into text-only models (A) and models that can handle image input (B).

files did not appear to substantially impact difficulty level, demonstrating generally strong image recognition capabilities for these models. For example, models attained close to a 50 percent accuracy level on the *Transit* suite on average.

We observed lower average accuracy in the *Clothes* suite, where average model performance was below 15 percent. Lastly, most models failed on the *Books* suite, consistently exhibiting close to 0 percent accuracy.

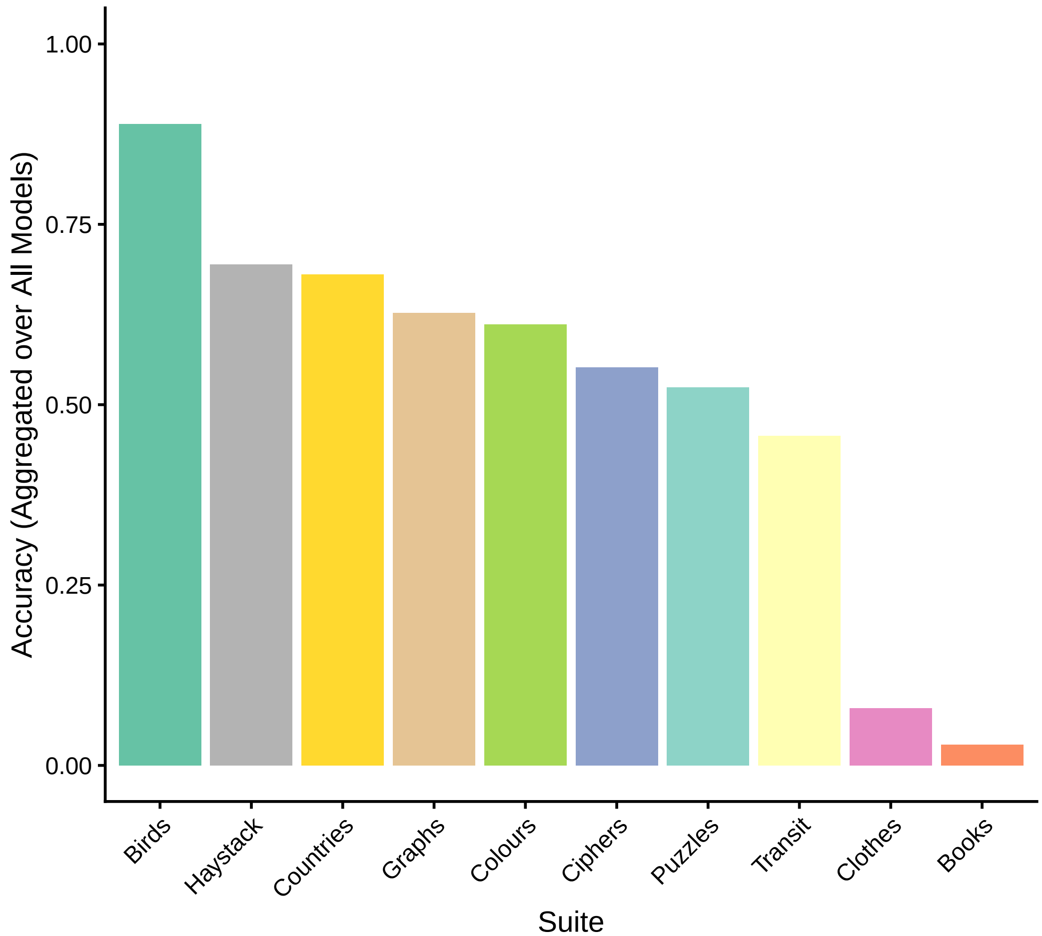

Figure 4: Performance by suite, averaged across all models and dates. Note that image-based suites were evaluated on a smaller set of models, as DeepSeek V4 Flash, DeepSeek V4 Pro, and Command R+ could not process image inputs.

4.3 Overall accuracy by run time and cost per model

The complete benchmark costs approximately $50 USD and takes up to one hour per day to run. We averaged out the accuracy of each model by its performance in all 11 testing suites. We visualize our results by plotting the outcome score against the average time to run each model (Figure 5) and the average cost of running each model (Figure 6).

The majority of models cost less than $0.01 dollars to run, with average accuracy ranging from 16.8 percent (Command R+, at $0.01 USD) to 71.2 percent (Deepseek V4 Flash, which was also the second-cheapest to run).

Models with highest accuracy included Deepseek V4 Flash and Deepseek V4 Pro, also at the lowest average cost per query. This could partly be attributed to Deepseek being evaluated on fewer suites as it does not support images as an input. Results also show that higher cost does not necessarily yield higher accuracy in most cases. For instance, Claude Fable 5 and Qwen 3.7 Plus achieve very similar accuracy levels (around 50 percent), but with a substantial difference in cost ($0.06 for Fable and $0.01 for Qwen 3.7 Plus).

GPT 5.6 Sol, GPT 5.5, and Claude Fable 5 were the most expensive models to run, costing between $0.04 and $0.06 per API run, respectively. These models produced responses at 67.7 percent (GPT 5.6 Sol), 65.6 percent (GPT 5.5), and 51.3 percent (Claude Fable 5) average accuracy, which were among the highest aggregate accuracies.

Figure 6 shows the average accuracy in model performance against the average response time for tasks, instead of costs. We put a five-minute time-out on each run, meaning if the model took more than five minutes to complete a task, we deemed it a failure. This was because the models did not take more than 1.5 minutes, on average, to respond.

On average, Command A+, Kimi K2.6, and Kimi K3 took the longest to produce outputs, at around 60 seconds, 70 seconds, and 90 seconds respectively. We also note that Kimi K2.6 frequently timed out, being unable to complete tasks in the allocated average time. In terms of average accuracy, Command A+ scored 36.9 percent, Kimi K2.6 scored 48.9 percent, and Kimi K3 scored 61.9 percent. The remaining models all took less than 30 seconds, on average, to respond, with varying accuracy.

Deepseek V4 Flash was the highest performing model, scoring 71.2 percent on average accuracy, while taking no more than 10 seconds, on average, to respond across tasks. While Command R+, Claude Haiku 4.5, GPT 5.4 mini, Claude Sonnet 4.6, and Claude Opus 4.8 all took less than 10 seconds to respond to prompts, average accuracy for each of these models was below 40 percent.

The remaining models took between 10 and 30 seconds to respond, with average accuracy ranging from 51.3 percent to 67.7 percent.

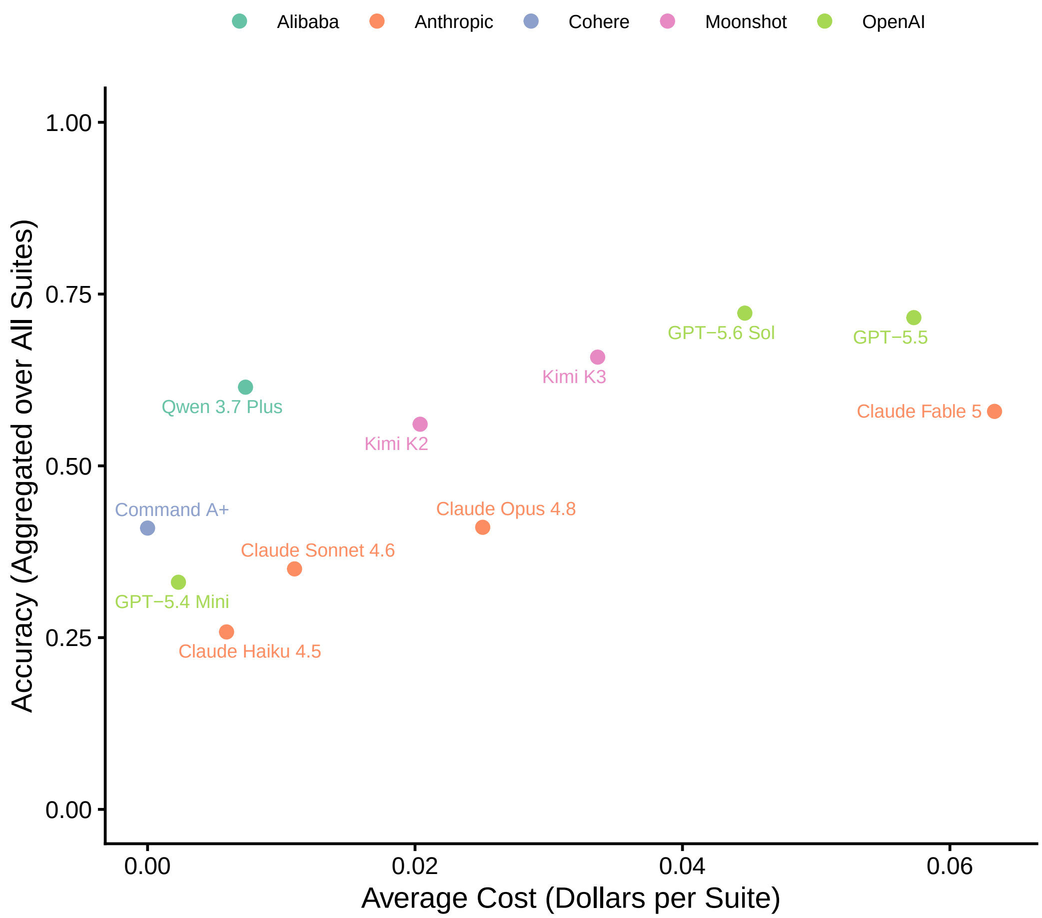

Figure 5: Relationship between accuracy and cost averaged over all suites for every model in our study, represented as points. Points are colour-coded according to their model providers. DeepSeek V4 Flash, DeepSeek V4 Pro, and Command R+ were omitted from this plot, as they were only evaluated on text-based suites.

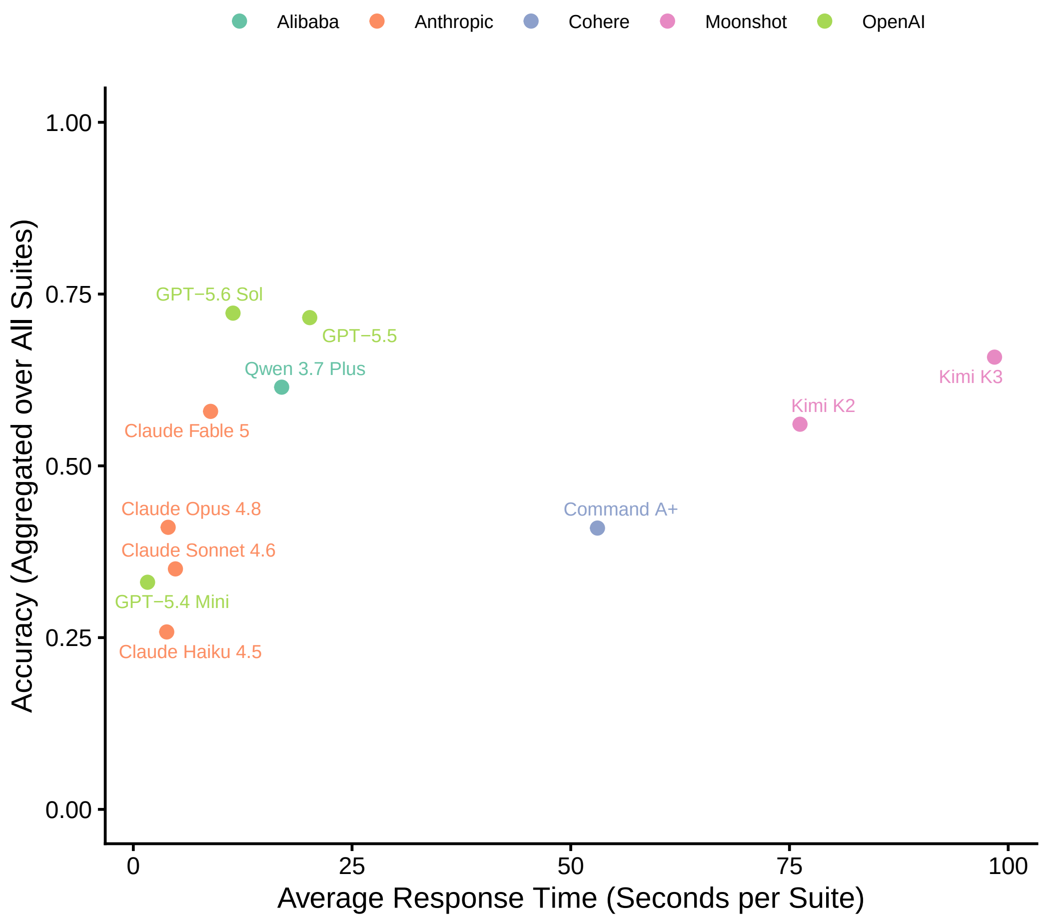

Figure 6: Relationship between accuracy and response time averaged over all suites for every model in our study, represented as points. Points are colour-coded according to their model providers. DeepSeek V4 Flash, DeepSeek V4 Pro, and Command R+ were omitted from this plot, as they were only evaluated on text-based suites.

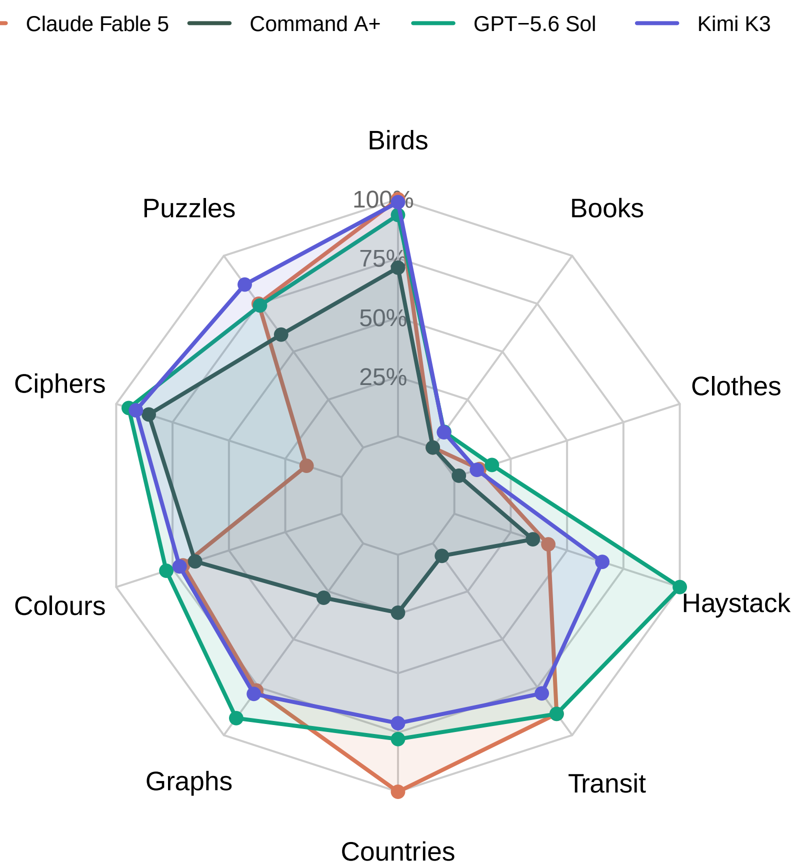

Figure 7: Radar plot displaying the performance of Claude Fable 5, Command A+, GPT-5.6 Sol, and Kimi K3 across each suite considered. Models are colour-coded according to their model providers.

4.4 Output token efficiency

The average accuracy of each model by the average amount of output tokens used is represented in Figure 8. We aimed to maximize token efficiency by explicitly instructing minimal communication, meaning we asked all models for direct responses without explanations or preambles.

Command A+ used the most output tokens by far (over 7,500), while producing an average accuracy of 36.9 percent. This finding demonstrates that using more output tokens does not necessarily result in higher accuracy. Command R+, Claude Haiku 4.5, GPT-5.4 mini, Claude Sonnet 4.6, and Claude Opus 4.8 all performed with less than 50 percent average accuracy, while each outputting less than 1,000 tokens. The lowest model accuracy was performed by Command R+ (16.8 percent). Claude Fable 5 and Qwen 3.7 performed with similar accuracy – 51.3 percent and 56.8 percent, respectively. However, Claude Fable 5 used less than 1,000 output tokens, whereas Qwen 3.7 used around 3,000.

The highest performing model was Deepseek V4 Flash, at 71.2 percent average accuracy, followed closely by Deepseek V4 Pro and GPT 5.6 Sol (both with 67.7 percent average accuracy), and GPT 5.5 (65.6 percent average accuracy). Each of these models used 2,000 or less output tokens.

4.5 Accuracy over time

Average accuracy over time for each model are presented in Figure 9. We note that Deepseek V4 Pro, Deepseek V4 Flash, and Command-R+ were excluded to ensure a fair comparison across models that were evaluated on all suites, rather than just the text-based ones.

We possess data for one week from July 20, 2026 to July 28, 2026. Model performance appears to fluctuate over time for all models. Some models like Claude Sonnet 4.6, Claude Opus 4.8 and Kimi K2 have relatively stable performances over time, with only mild deviations from their center. On the other hand, models like Command A+, GPT-5.6 Sol and Qwen 3.7 Plus have much more variable performance over time.

Figure 9 also shows what appears to be 3 almost entirely non-overlapping groups of models by performance. The first group seems to lie between 60 and 75 percent accuracy (Claude Fable 5, GPT-5.5, GPT-5.6 Sol and Kimi K3), the second lies between 45 and 60 percent performance (Claude Sonnet 4.6, Kimi K2, Qwen3.7 Plus, and Claude Opus 4.8), and the last lies between 20 and 35 percent performance (Command A+, GPT 5.4 Mini and Claude Haiku 4.5). This is interesting because it shows a different picture compared to Figure 3, which only showed models by their average accuracy on the benchmark overall. Figure 9 shows us why Figure 3 may vary over time, since model performance is not constant every day.

For instance, on July 27, Command A+ performed worse than Claude Haiku 4.5 and GPT-5.4 Mini, when it had performed better than both of them the day before. On July 26, GPT-5.6

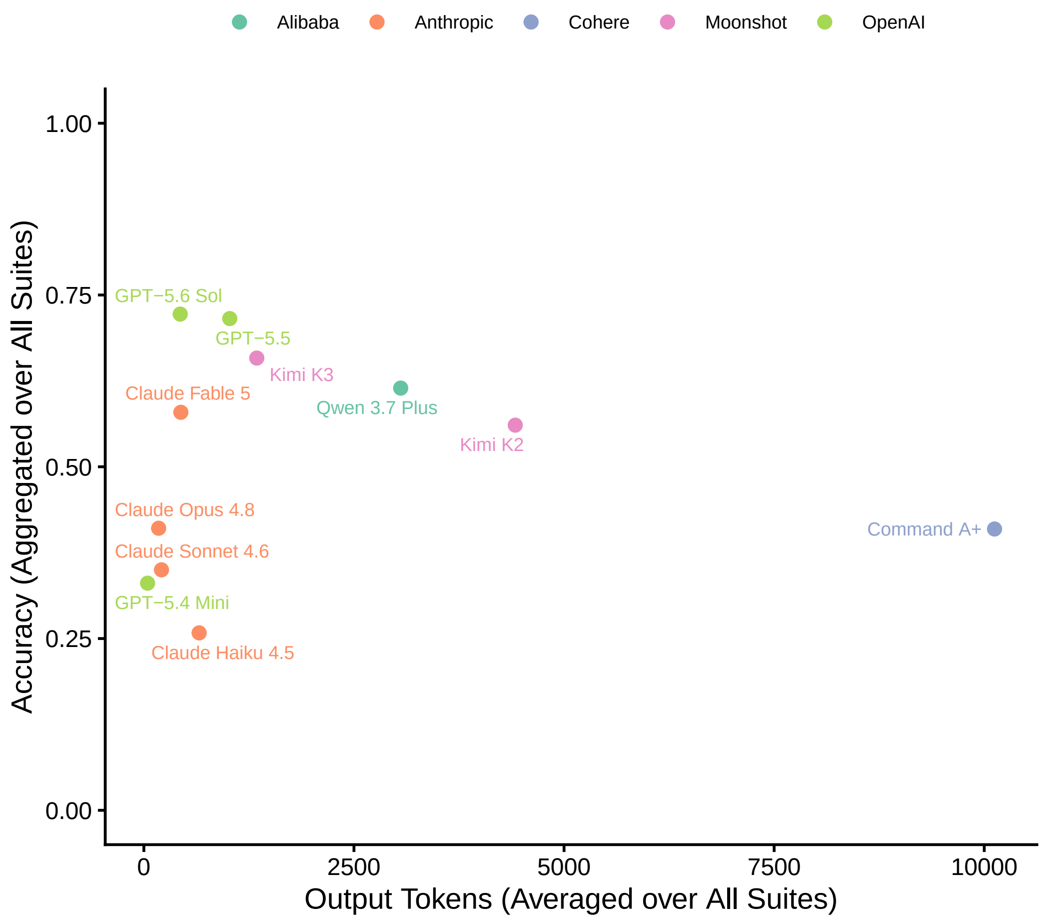

Figure 8: Relationship between accuracy and number of output tokens used averaged over all suites for every model in our study, represented as points. Points are colour-coded according to their model providers. DeepSeek V4 Flash, DeepSeek V4 Pro, and Command R+ were omitted from this plot, as they were only evaluated on text-based suites.

Sol achieved an all-time high performance of almost 75 percent, after being outperformed by GPT-5.5 and Claude Fable 5, just days before and being outperformed again the day after. This demonstrates the variability in model performance on the same tasks over time, supporting the importance of capturing and reporting daily evaluations.

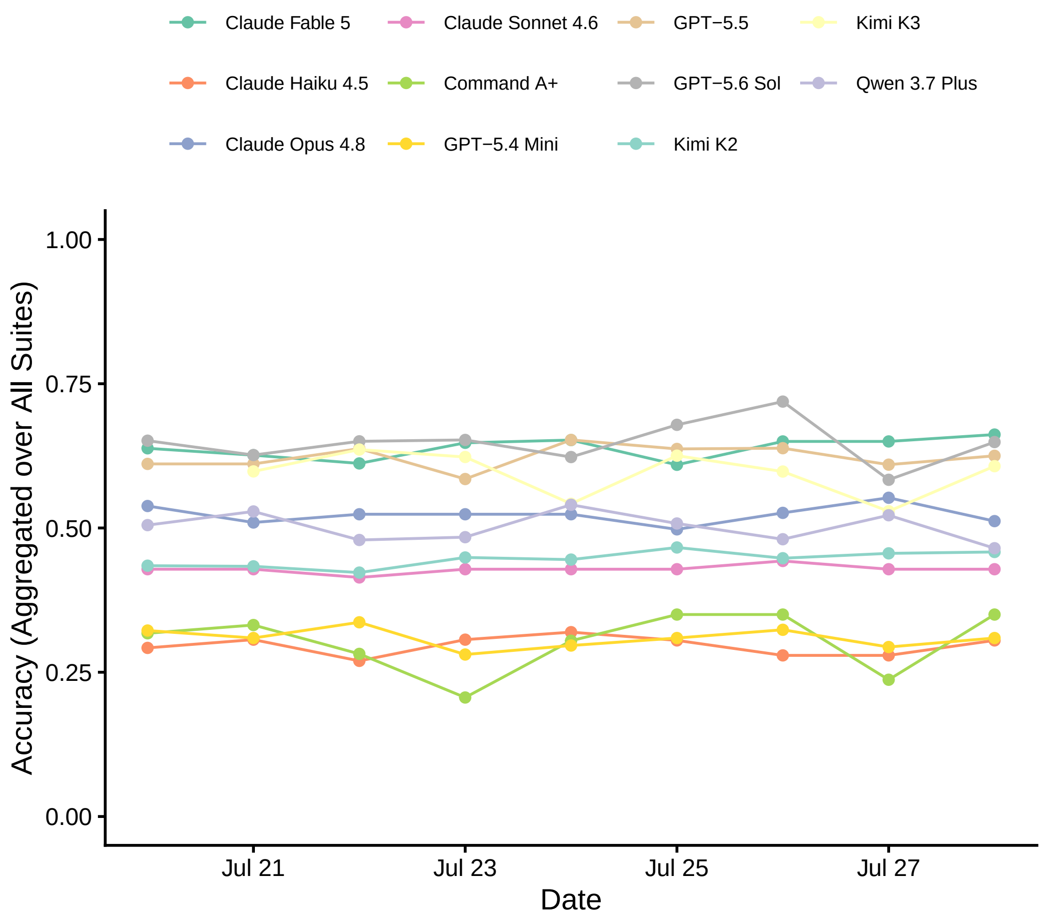

Figure 9: Time series of model performances from July 20, 2026 to July 28, 2026. Model performance is the average accuracy scored by the model across all suites. DeepSeek V4 Flash, DeepSeek V4 Pro, and Command R+ are omitted from this plot since they were not evaluated on image-based suites.

4.6 The `GardenBench` live dashboard

Our website[4] provides an overview of each model's performance on each suite, representing a breakdown across models and their specific capabilities tested by the suites, compared to just looking at aggregated accuracy across all suites. A radar plot is presented in Figure 7. The graph displays the average results of all the iterations of the benchmark over time. In other words, it averages the results we obtain by running `GardenBench` every day on all 17 models for all 11 suites. In this section, we explore results from 8 days of running GardenBench. Results described in this section were retrieved on July 28, 2026 and can be found alternatively in Appendix B.

The *Birds* suite, which appeared the least challenging one as per Figure 4, had three models (Fable, Opus and Kimi K3) achieve perfect accuracy across all tests of the suite. That is, they identified the birds from the image correctly 100 percent of the time. At 90+ percent accuracy we find Kimi K2.6, GPT-5.5, GPT-5.6 Sol and Sonnet 4.6, followed by models at 80+ percent accuracy like Qwen 3.7 and Haiku 4.5. Even the relatively less accurate models like Command A+ and GPT-5.4 Mini still achieved an accuracy of over 70 percent.

In the *Transit* suite, no model achieved 100 percent accuracy. Top performing models at over 90 percent accuracy included Kimi K3, GPT-5.6 Sol, GPT-5.5 and Claude Fable 5. In this suite, following models performed at significantly lower accuracies, with Qwen 3.7 attaining a 44 percent accuracy, Kimi K2.6 26 percent and Sonnet 4.6 16 percent. Remaining models attained less than 10 percent accuracy. These results coincide with the average accuracy on the transit suite in Figure 4, either models performed with really high or really low accuracy resulting in an average of around 50 percent in the entire suite. We notice that more recent models are the ones performing better at this task, demonstrating efforts to improve spatial reasoning capabilities in newer models.

The *Haystack* suite, which tested extraction of a specific string within a large corpus of text, had multiple models achieve perfect accuracy, namely GPT 5.4 mini, GPT-5.5 and GPT-5.6 Sol. These were followed closely by DeepSeek V4 Flash and Qwen 3.7, which scored above 90 percent. Contrarily, there are also multiple models that attain less than 50 percent accuracy — Sonnet 4.6, Haiku 4.5, Fable, Opus and Command A+. Models in between include DeepSeek V4 Pro, Kimi K3 and Command R+ achieving accuracies between 60 and 70 percent.

The *Graphs* suite had no model that achieved perfect accuracy. GPT-5.5 and GPT-5.6 Sol achieved over 90 percent accuracy, followed by Kimi K3 at 80 percent. Qwen 3.7, Fable, Kimi K2.6 and Opus, achieved between 60 and 70 percent. Lastly, models like Sonnet 4.6, Command A+ and Haiku 5.4 attained less than 40 percent accuracy.

Similarly, the *Puzzles* suite has no model performing perfectly. High-performing models included GPT-5.5 and GPT-5.6 Sol, at 90 percent accuracy. Kimi K3, Qwen 3.7 Plus, Fable and Opus all achieve accuracies between 70 and 80 percent. However, similar to the *Transit* suite, these

[4] https://gardenbench.ai/

high accuracies are offset by low performances across models like GPT 5.4 Mini at 8.3 percent accuracy and Sonnet 4.6, Haiku 5.4, Opus 4.8 and Command R+, resulting in the average accuracy presented in Figure 4.

The *Countries* suite, in which models achieved the third highest overall accuracy as in Figure 4, had a few models attain perfect accuracy. This included Fable, Opus and Sonnet, followed by Kimi K2.6 at 87.9 percent accuracy and GPT-5.6 Sol at 75.8 percent accuracy. Although some models performed really well here, others performed with much lower accuracy, mainly Qwen 3.7+ (33.3 percent accuracy), Command A+ (27.3 percent accuracy) and GPT-5.4 Mini (24.2 percent accuracy). Notably, Command R+ attains once again a zero percent accuracy.

The *Colours* suite showed a closer performance across models. 7 models out of 17 achieved accuracies between 60 and 80 percent, with no model achieving perfect accuracy. Outstanding models include Kimi K3 (79.2 percent), GPT-5.6 Sol (75 percent) and GPT-5.5 (74.6 percent). Claude Sonnet 4.6, Claude Haiku 4.5, and Kimi K2 achieved accuracies of less than 50 percent, and again Command R+ had a zero percent accuracy.

The *Clothes* suite appears extremely challenging for models relative to other suites. Qwen 3.7 achieved the highest accuracy across all models at 56 percent, followed by Fable and GPT-5.6 Sol which achieved accuracy of 13.3 percent. Remaining models scored below 10 percent accuracy.

Lastly, the *Ciphers* suite had four models achieve perfect accuracy: Deepseek V4 Flash, Deepseek V4 Pro, GPT-5.5 and GPT-5.6 Sol. Two models achieved over 90 percent accuracy (Kimi K3 and Kimi K2) and two models achieved over 85 percent (Qwen 3.7 and Command A+). Additionally, Haiku 5.4, Opus, Sonnet and Command R+ all attained zero percent accuracy.

We note that results are subject to change as we update the website with more iterations of `GardenBench` being run and added to the website. Additionally, current results in Figure 4 do not exclude models that do not support image inputs such as DeepSeek V4 Flash and DeepSeek V4 Pro. As a result, current average accuracies may be skewed. The fact that these models are not tested on suites that take image as inputs could also impact our ability to compare their aggregate accuracy with other models' as they are not tested on the same number of suites.

5 Discussion

5.1 Differentiating `GardenBench` from existing benchmarks

In conceptualizing `GardenBench`, we addressed two main shortcomings in previous methods of LLM benchmarking. First, `GardenBench` is publicly accessible, with conceivable results posted daily[5]. This addresses concerns (for example, Guha et al. 2026; Koch et al. 2021) that LLM benchmarks tend to target audiences of niche scientific communities, with complex and

[5] https://gardenbench.ai/

highly specialized evaluations. Moreover, benchmarks are often conducted or published by model providers, for example, Anthropic (2026a)'s `BioMysteryBench`. Studies find that model providers tend to uniquely publish benchmarks on which their models performed well, choosing not to disclose less successful performances (Cheng et al. 2025; Baack et al. 2026; Mitchell et al. 2019). We believe that LLM evaluation mechanisms should be as broadly accessible as LLMs themselves, keeping pace with the widespread use of LLMs for general, everyday tasks by the vast public. By publishing all of our results to a live and public dashboard, we indiscriminately show LLM capabilities in a way that is meaningful and useful to broad LLM consumers.

Second, LLM benchmarks are often expensive and time-consuming, resulting in infrequent testing (Polo et al. 2024). Figure 5 and Figure 6 respectively show that, on average, API runs for `GardenBench` cost no more than $0.06 USD and took no longer than 90 seconds per run (except for Kimi K2.6, which timed out on many occasions). In total, the complete benchmark costs approximately $50 USD and takes up to one hour per day to run. Systematically and consistently running our tests at the same time each day of the week allows us to achieve a high level of reliability and scalability in our results.

5.2 Takeaways from model performance

Figure 4 allowed us to observe areas where model performance was relatively successful, and others where LLMs appeared to be underdeveloped. Specifically, string matching, image recognition (from image-capable models), visual perception, and general knowledge appeared to be more developed qualities of LLMs, exhibited by overall performance in tasks from the *Birds*, *Haystack*, *Countries*, *Graphs*, and *Colours* suites. The tested models were slightly less successful in *Ciphers* and *Puzzles* suites, indicating that logical and algorithmic reasoning are less developed areas. Lastly, relatively low accuracy in the *Transit* (48 percent), *Clothes* (10 percent), and *Books* (0 percent) suites suggest that LLMs generally struggle with spatial reasoning questions, contextual understanding, and information extraction.

Our benchmark construction process also revealed how prompt engineering may impact model performance. In developing testing suites, we observed that model performance improved significantly when models were asked to follow a specific prompt structure, for instance, in terms of capitalization and punctuation. Asking for one-word responses, comma-separated lists, or binary answers also allowed us to simplify our evaluation method while preserving reliability and depth of analysis.

Secondly, our results for model performance account for API run times and costs, exhibiting trade-offs associated with achieving accuracy.

From the 4 text-based suites, DeepSeek V4 Flash and DeepSeek V4 Pro were the highest performing models, while also being among the most cost-efficient (see Figure 5). DeepSeek V4 Flash also took relatively little time to produce responses (approximately 10 seconds), while DeepSeek V4 Pro took almost 30 seconds. Other relatively high performing models also took less than 25 seconds to output responses, namely GPT-5.6 Sol and GPT-5.5, which were evaluated on all 11 test suites. This suggests that LLM users can receive accurate responses without waiting excessively for an answer. We also demonstrate, to the benefit of other LLM researchers, that LLMs can be tested robustly without the expense of costly and long run times.

Lastly, results from per suite performance clearly indicate the tasks where models struggle most as well as which specific models. Generally, models performed with low accuracy on suites like *Clothes* and *Books* which were both based on author-collected images, in the case of clothes images of a closet and in the case of books images of a bookshelf. In the case of the *Clothes* suite, we noticed the task at hand, counting the number of items folded in the closet may have been difficult for the models since some clothes were very thin, dark or looked very similar to each other and may not have been counted correctly. However, this is a task that can be easily performed by a human.

In the case of the *Books* suite, where models were tasked with naming the titles and authors of books in the bookshelf, they also performed with low accuracy. On many occasions, models were able to correctly identify the book title but failed to identify the author. It did not return an author name at all, and so it was given incorrect for that run.

Another suite containing author-collected images was the *Colours* suite. These were images of paintings mixed in a paint palette by one of the authors. Even though the images were also author-collected, models performed much better in terms of accuracy compared to the *Books* and *Clothes* suite. We believe that the fact that this prompt provided the model with some options to pick from made the models achieve such high accuracies compared to if they were given no options. For instance, a model would be asked to identify the three colors used in the palette, and given some options to pick from. This likely boosted model performance and we suspect that implementing this for the *Clothes* and *Books* suites would possibly boost accuracy as well.

Suites that involve internet-sourced images such as *Birds*, *Countries* and *Graphs*, performed better than suites involving author-collected images described above. Although it is unclear whether the quality of the images had an effect on the accuracies of the corresponding suites, it may have contributed to the lower performance on author-collected images compared to internet-sourced images. This may have also been due to the difficulty of the tasks themselves.

The most common error in the *Birds* suite was that most models correctly identified the Genus of the bird, but got the Species wrong. This resulted in a completely incorrect answer, since they didn't consider partial correctness. In the *Countries* and *Graphs* the output from the model was entirely incorrect. For instance, in the *Countries* suite the main error was responding with the incorrect country. Conversely, there was variety of errors in the *Graphs* suite; in some cases the model was entirely incorrect by providing the wrong equation, in other cases the models attempted to match the shape of the image, but failed to plug in the numbers to actually check whether it matched, resulting in the incorrect function with a similar shape to the ground truth.

In terms of model specifics, Anthropic models returned reasoning frequently, despite being clearly instructed not to include pleasantries or additional text in their responses. On occasions where models returned additional text, even if the answer to the task was correct, answers were coded as incorrect. This is because our benchmark is not only checking that the models can perform the task correctly by providing the right answer, it is also checking whether the models are able to follow specific instructions by returning output that follows a structured schema (no additional text aside from the answer and specific answer format provided in the prompt itself).

5.3 Limitations and future directions

One limitation of our benchmark is that it involves text- and image-based prompting, even though 3 of our tested models (DeepSeek V4 Flash, DeepSeek V4 Pro, and Command-R+) were independently incapable of image processing. It is possible that the accuracy of these models may be overstated as they were evaluated on a reduced set of testing suites (4 suites instead of 11). Moreover, some testing suites varied in the number of tests included, which also could have influenced average accuracy representations.

In addition, our tests restricted models from web searching in order to evaluate the raw capabilities of LLMs. However, Internet information retrieval is an important feature of LLMs that is necessary for common general-use prompts. Therefore, future work could conduct similar general-purpose tests with extracting knowledge from the web as an option for LLMs. This would allow researchers to investigate whether model performance improves when supporting resources are used, and to what degree.

As LLM capabilities continue to progress, benchmarks should continuously expand evaluation to the newest features of LLMs, such as image and audio processing, as well as increasingly complex concepts and prompts. One way to do this is by having varying difficulty levels within one benchmark. Incorporating tasks with graduated complexity is another way to account for modest progress. In general, we encourage future benchmarks to continue evaluating LLM capabilities across general-use tasks that are relevant to broad communities, and that are relatively unaccounted for in existing benchmarks.

6 Conclusion

This paper introduced `GardenBench`, offering an innovative approach to measuring LLM capabilities across various general use cases. We join emerging calls to reduce the gatekeeping of LLM evaluation and help address this concern, primarily through the publication of our live dashboard[6]. The main objective of `GardenBench` is to make LLM evaluation publicly accessible, in a transparent and trustworthy manner. Granting broad access to such rankings allows us to

[6] https://gardenbench.ai/

understand the most effective ways to implement LLMs in our daily lives, as is already being done.

We also stress the importance of having benchmarks that can be run in a lightweight manner, such that they can be run frequently to capture model performance over time and adapted to new models effectively and immediately once they are released.

A Prompt details

Below we provide more detail on the suites of `GardenBench` including a description of the tasks tested and an example of the prompts.

A.1 Birds

The Birds test suite evaluates the capacity of Large Language Models to process image data to perform species identification, where the model must identify the primary calling species using its scientific name. Performance across all experimental scenarios is automatically evaluated via exact-match string parsing against ground-truth files, tracking responses against `birds/01-ground_truth.txt` for the classification task and `birds/01-ground_truth.txt` for the species identification task.

Example prompt:

> This is an image of a specific species of bird. Respond with the scientific name of the species. Notes: Please respond in plaintext. Do not bold or italicize your answers. Place a space between the genus and the species. Do not include punctuation. RESPONSE STRUCTURE: [Genus] [species]
>
> Insert `birds/01-additional_file_01.png`

Expected output:

> Troglodytes hiemalis

A.2 Books

The Books test suite evaluates an LLM's capability to parse complex visual data and extract structured information from images. The dataset consists of photographs of unique bookshelves of varying sizes and book quantities, captured directly by one of the authors with ground-truth labels established manually. Designed to imitate tasks performed in professional library settings, the suite features varying levels of difficulty. It tests the model's ability to answer direct visual questions, ranging from simple tasks like identifying the first book on a shelf to listing out the entire bookshelf. It has also been provided a strict formatting structure, in order to enable the comparison of its result to a ground truth. These benchmarks require the model to parse the image accurately and combine visual inputs with external context to generate detailed lists of the volumes present.

For example, we have `books/01-additional_file_01.png` and we ask the model the following prompts as examples.

Example prompt:

> What is the first book on the left?
>
> Insert `books/01-additional_file_01.png`
>
> Please provide a list of the books.

Expected output:

- Alammar, Jay, and Maarten Grootendorst, "Hands-On Large Language Models"
- Galsworthy, John, "The Man of Property"
- Galsworthy, John, "The Silver Spoon and Passers By"
- Galsworthy, John, "The White Monkey and A Silent Wooing"
- Galsworthy, John, "Swan Song"
- Unknown

A.3 Ciphers

The Ciphers test suite evaluates an LLM's capacity for algorithmic reasoning, character mapping adherence, and logical computation by requiring it to decode hidden words using fixed character-level transformation rules. The benchmark prevents models from relying on guessing or predictive patterns by making target outputs lack semantic or contextual cues. This could mean making a spelling error in the target word, and evaluating whether the LLM correctly computes it or instead incorrectly outputs the error-free word. In a baseline scenario, models are evaluated using a zero-shot prompt with an explicit output constraint, yielding a target string such as "BENCHMARK". To test the limits of character-by-character logical execution, the suite introduces more complex extensions, including multi-step transformations such as $f(x)=(ax+b)(\mathrm{mod}26)$ and dynamic shifts based on character position. Performance is assessed via exact-match evaluation, where the model's output is stripped of all whitespace and compared directly against the ground-truth target string.

Example prompt:

> Decode the hidden word using the given transformation rule. Alphabet mapping: A=0, B=1, ..., Z=25. Rule: $f(x)=25-x$, where x represents the integer value of each letter.
>
> Encoded Word: YVMXSNZIP.

Output Constraint: Return only the decoded word.

Do not include any explanation, preamble, or punctuation"

Insert `ciphers/01-additional_file_01.png`

Expected output:

BENCHMARK

A.4 Clothes

The Clothes test suite benchmarks an LLM's capacity for object detection, visual counting, and classification in a singular image. The dataset comprises ten photographs of a single closet, capturing changes across instances, such as the removal or rearrangement of specific items. All images were captured by one of the authors and are utilized for the benchmark with their consent. This design evaluates the model's image recognition counting accuracy. Evaluation is conducted using zero-shot prompting strategies.

Example prompt 1:

Task: Count the total number of items visible in the closet.

Constraints:

1. Provide only the final count as a single integer.
2. Do not include any units, labels, introductory text, pleasantries, or explanations (e.g., return "14", not "14 items").

Response Format: [Number]

Insert `clothes/01-additional_file_01.png`

Expected output:

55

Model outputs are assessed via exact-match string parsing, comparing responses directly against the corresponding verification targets in closet/01_ground_truth.txt.

A.5 Colours

The Colours test suite introduces a benchmark that evaluates a model's visual perception and reasoning. Rather than using standardized colour codes, the dataset consists of photographs of physical paint palettes containing mixed acrylic paints. This design assesses the model's ability to handle artistic challenges and parse blends and textures of colours.

The model is first prompted with a palette image and asked to identify the kind. It is also prompted to list specific colours present. All images were captured by one of the authors and are utilized for the benchmark with their consent. Performance is evaluated through an exact-match. The model's generated color list is compared directly against a ground-truth list of the exact color names, alphabetized for consistency.

Example prompt 1:

> What kind of paint was used in this palette? Please respond with one word.
>
> Insert `colours/01-additional_file_01.png`

Expected output:

> Acrylic

Example prompt 2:

> There are six distinct colours used to create the colour in the attached paint palette.
>
> The following is a list of the possible colours: Black, Brown, Dark Green, Forest Green, Light Blue, Prussian Blue, Turquoise, White, Yellow, Yellow Ochre.
>
> From this list, please return the six colours that were used on the palette in a comma-separated list, with capitalized colour names, and in alphabetical order.
>
> Insert `colours/12-additional_file_01.png`

Expected output:

> Dark Green, Forest Green, Prussian Blue, Turquoise, White, Yellow Ochre

A.6 Countries

The Countries test suite evaluates an LLM's visual pattern recognition and geographic knowledge retrieval by challenging it to identify a specific country based on the outline of its geographic boundary. Inspired by the popular geography game Worldle, this benchmark forces models to process spatial contours and map those shapes to geographic data without relying on text labels, regional context, or surrounding borders. All images of country outlines were sourced from Canva under the Free Plan, and provided to the model as .png files. Evaluation is conducted across zero-shot and one-shot scenarios. In the zero-shot setting, the model is provided with a geographic file and a direct identification directive.

Zero-shot example:

Insert `countries/01-additional_file_01.png`

What country is this? Only return its name.

Expected output:

Suriname

One-shot scenario:

For example, this image: `countries/11-additional_file_01.png`

Returns: Albania

What country is this? Only return its name.

Insert `countries/01-additional_file_01.png`

Expected output:

Suriname

A.7 Graphs

The Graphs test suite measures geometric reasoning, visual data interpretation, and image-generation capabilities by prompting models to perform math operations on plots. Specifically, the benchmark requires the model to interpret the visual curve of a function and generate the corresponding graph of its first derivative, or conversely, to get the original function from a provided derivative graph. This task tests the model's capacity to visually identify mathematical knowledge like inflection points, local extrema, and asymptotic behavior, and translate these into an accurate representation. The suite uses zero-shot prompting, where the configuration asks for the mathematical graph once provided the image input. Evaluation will then involve testing the LLM as a judge, and determine through a Yes/No if the images are identical.

Example prompt:

> Task: Analyze the provided graph of the function f(x) and output the equation of its derivative function, f'(x), as text.
>
> Here is the graph of f(x): `graphs_input_10.png`.
>
> Constraints:
>
> 1. The output must not be Latex, return simple text. That means, do not return (f'(x) = 2x), return f'(x)=2x.
>
> 2. Do not include any spaces. That means, do not return f'(x) = 2x, return f'(x)=2x.
>
> 3. Do not include any text explanations, markdown commentary, or pleasantries in your response.
>
> Output only the equation.

Expected output:

> f'(x)=2x

A.8 Puzzles

The Puzzles test suite benchmarks an LLM's capacity for context-window rule adherence, logical reasoning, and multi-step deduction by evaluating its performance on Knights and Knaves logic puzzles (Smullyan 1981). In these scenarios, a set of characters are designated as either knights (who strictly speak the truth) or knaves (who strictly lie) and make self-referential statements about one another's identities. These puzzles are deterministic and solvable via formal inference engines. We measure how reliably LLMs deduce pure logic compared to standard computation. The dataset comprises 10 structurally distinct puzzles featuring scaled complexities ranging from 3 to 6 characters and diverse statement types. The puzzles and corresponding solutions were programmatically generated and verified by adapting the open-source TruthQuest framework (Mondorf and Plank 2024).

Example prompt:

In this puzzle, every character is either a knight or a knave: knights only tell the truth, and knaves only lie. Your objective is to determine the identity of every character based on what they say.

You are met with 3 people, who say the following things:

A: “B is a liar.”,

B: “C is a liar.”,

C: “A is a truth-teller and B is a truth-teller.”

In your answer, you must clearly state the identity of each character by following the format:

CONCLUSION:

A: [Knight/Knave]

B: [Knight/Knave]

C: [Knight/Knave]

Expected output:

CONCLUSION:

A: Knave

B: Knight

C: Knave

A.9 Transit

Example Prompt:

TASK: Here is a map of a city’s transit system. Respond with the list of stops between the following stops, separated by commas (including the start and end stops themselves):

> Insert `01-additional_file_01.png`
>
> START: Lawrence
>
> END: Rosedale
>
> NOTE: Decide on the path that includes the least amount of stops. Respond with only the list, nothing else.

Expected output:

> Lawrence, Eglinton, Davisville, St Clair, Summerhill, Rosedale

A.10 Ascii Haystack and Haystack

The Haystack Suite tests whether the model can identify a target string from within a large corpus of text. The main difference between Ascii Haystack and Haystack is that Ascii Haystack embeds the relevant information within random letters and symbols, whereas Haystack embeds random letters and symbols within coherent prose.

Example Prompt for Ascii Haystack:

> Embedded in this text is a fact about a person. Return this fact, and nothing else.
>
> The text is as follows:
>
> fivbifbkendjeriugbfuhvndkoerigrtiy4855836405ibe irhbein29448 tgriufbvirtub5837 54vifuberygb8475tg856f8erb v856fb8erbiruf beirubiurferuf857tgRohan's middle name is Peter. 874fbrivbitgb 4587836e2938r058g8rt7vbubveirybg yfverb948bg8berbgirugbirugb59769eruvbivbivbivriuberb irgbtirberigbegbeirufbeirgbitygbdtghrt7vbubveirybgyfverb948 bg8rt7vbubveirybgy fverb948bg8.

Expected output:

> Rohan's middle name is Peter.

Example Prompt for Haystack:

> Embedded in this text is a unique identifier, which is a UUID. Your task is to find and return this UUID.

The text is as follows: JANE EYRE AN AUTOBIOGRAPHY by Charlotte Brontë ILLUSTRATED BY F. H. TOWNSEND London SERVICE & PATON 5 HENRIETTA STREET 1897 The Illustrations in this Volume are the copyright of SERVICE & PATON, London TO W. M. THACKERAY, ESQ., This Work IS RESPECTFULLY INSCRIBED BY THE AUTHOR PREFACE A preface to the first edition of "Jane Eyre" being unnecessary, I gave none: this second edition demands a few words both of acknowledgment and miscellaneous remark. My thanks are due in three quarters. To the Public, for the indulgent ear it has inclined to a plain tale with few pretensions. To the Press, for 1eb258ef-1bb6-4c9c-8cf8-125bbb0bb08c the fair field its honest suffrage has opened to an obscure aspirant. To my Publishers, for the aid their tact, their energy, their practical sense and frank liberality have afforded an unknown and unrecommended Author. The Press and the Public are but vague personifications for me, and I must thank them in vague terms; but my Publishers are definite: so are certain generous critics who have encouraged me as only large-hearted and high-minded men know how to encourage a struggling stranger; to them, i.e., to my

Expected Output:

1eb258ef-1bb6-4c9c-8cf8-125bbb0bb08c

A.11 Suites in progress

A.11.1 Self Check

The Self Check test suite evaluates the model's ability to answer questions about the environment in which it is operating, for instance, "What date is it today?". This test encompasses a variety of text prompts and tests the model's real time knowledge retrieval and system awareness.

Example prompt:

What model are you? Return your API model slug only.

B Additional tables and graphs

Table A1: Full table of accuracies by models and suites. Accuracies are averaged across time. We note that Deepseek V4 Flash, Deepseek V4 Pro, and Command R+ were only evaluated on text-based suites.

	Suite							
Model	**Birds**	**Books**	**Ciphers**	**Clothes**	**Colours**	**Countries**	**Puzzles**	**Graphs**
Claude Fable 5	1	0	0.156	0.111	0.7	1	0.75	0.77
Claude Haiku 4.5	0.8	0	0.011	0.044	0.48	0.55	0	0.2
Claude Opus 4.8	1	0	0	0.056	0.64	1	0	0.61
Claude Sonnet 4.6	0.9	0	0	0	0.5	1	0	0.44
Command A+	0.71	0	0.856	0.02	0.65	0.24	0.589	0.28
Command R+	NA	NA	0	NA	NA	NA	0	NA
DeepSeek V4 Flash	NA	NA	1	NA	NA	NA	0.833	NA
DeepSeek V4 Pro	NA	NA	1	NA	NA	NA	0.917	NA
GPT-5.4 Mini	0.74	0	0.078	0	0.58	0.2	0.093	0.56
GPT-5.5	0.91	0.081	1	0.045	0.73	0.73	0.796	0.88
GPT-5.6 Sol	0.93	0.082	0.944	0.167	0.78	0.78	0.741	0.91
Kimi K2	0.9	0	0.933	0	0.4	0.92	0.87	0.68
Kimi K3	0.99	0.08	0.912	0.1	0.72	0.71	0.85	0.78
Qwen 3.7 Plus	0.89	0.072	0.833	0.333	0.54	0.36	0.898	0.79

Table A2: Full table of exact model API strings.

Provider	Model Name	API String
Alibaba Cloud (via Fireworks)	Qwen 3.7 Plus	accounts/fireworks/models/qwen3p7-plus
Anthropic	Fable 5	claude-fable-5
Anthropic	Haiku 4.5	claude-haiku-4-5
Anthropic	Opus 4.8	claude-opus-4-8
Anthropic	Opus 5	claude-opus-5
Anthropic	Sonnet 4.6	claude-sonnet-4-6
Cohere	Command-A+	command-a-plus-05-2026
Cohere	Command-R+	command-r-plus-08-2024
Deepseek	V4 Flash	deepseek-v4-flash
Deepseek	V4 Pro	deepseek-v4-pro
Google	Gemini 3.1 pro	gemini-3.1-pro-preview
Google	Gemini 3.5 flash	gemini-3.5-flash
Moonshot AI (via Fireworks)	Kimi K2.6	accounts/fireworks/models/kimi-k2p6
Moonshot AI (via Fireworks)	Kimi K3	kimi-k3
OpenAI	GPT 5.4 mini	gpt-5.4-mini
OpenAI	GPT 5.5	gpt-5.5
OpenAI	GPT 5.6 Sol	gpt-5.6-sol

References

Ahn, Jihyun Janice, and Wenpeng Yin. 2025. *Prompt-Reverse Inconsistency: LLM Self-Inconsistency Beyond Generative Randomness and Prompt Paraphrasing*. https://arxiv.org/abs/2504.01282.

Anthropic. 2026a. *Evaluating Claude's Bioinformatics Research Capabilities with BioMystery-Bench*. https://www.anthropic.com/research/Evaluating-Claude-For-Bioinformatics-With-BioMysteryBench.

Anthropic. 2026b. *System Card: Claude Opus 5*. https://www-cdn.anthropic.com/b514064af1408018e64b1ad24e7d5e75850b4ffd/Claude%20Opus%205%20System%20Card.pdf.

Baack, Stefan, Maty Bohacek, and Christo Buschek. 2026. "Unsteady Metrics and Benchmarking Cultures of AI Model Builders." (New York, NY, USA). https://doi.org/10.1145/3805689.3812240.

Bell, Robert M., Yehuda Koren, and Chris Volinsky. 2010. "All Together Now: A Perspective on the Netflix Prize." *CHANCE*, no. 1: 24–29. https://doi.org/10.1080/09332480.2010.10739787.

Bommasani, Rishi, Drew A. Hudson, Ehsan Adeli, et al. 2022. *On the Opportunities and Risks of Foundation Models*. https://arxiv.org/abs/2108.07258.

Bommasani, Rishi, Kevin Klyman, Shayne Longpre, et al. 2023. *The Foundation Model Transparency Index*. https://arxiv.org/abs/2310.12941.

Brown, Tom B., Benjamin Mann, Nick Ryder, et al. 2020. "Language Models Are Few-Shot Learners." *CoRR* abs/2005.14165. https://arxiv.org/abs/2005.14165.

Chen, Lingjiao, Matei Zaharia, and James Zou. 2024. "How Is ChatGPT's Behavior Changing Over Time?" *Harvard Data Science Review* 6 (2). https://hdsr.mitpress.mit.edu/pub/y95zitmz/release/2.

Chen, Simin, Yiming Chen, Zexin Li, et al. 2025. "Benchmarking Large Language Models Under Data Contamination: A Survey from Static to Dynamic Evaluation." In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, edited by Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng. Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.emnlp-main.511.

Cheng, Zerui, Stella Wohnig, Ruchika Gupta, et al. 2025. "Benchmarking is Broken – Don't

Let AI be its Own Judge." *Advances in Neural Information Processing Systems* 38. https://proceedings.neurips.cc/paper_files/paper/2025/hash/693e00827fd44bdfca210801fe1e6439-Abstract-Position_Paper_Track.html.

Chiang, Wei-Lin, Lianmin Zheng, Ying Sheng, et al. 2024. *Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference*. https://arxiv.org/abs/2403.04132.

Cui, Jiaxin, and Rohan Alexander. 2026. *Same Prompt, Different Outcomes: Evaluating the Reproducibility of Data Analysis by LLMs*. https://arxiv.org/abs/2602.14349.

De Angelis, Inessa. 2026. "Using GitHub Actions for Computational Communication Research." *SocArXiv*, ahead of print. https://doi.org/10.31235/osf.io/uqf6n_v1.

Epoch AI. 2026. *Data on AI Capabilities and Benchmarking*. Epoch AI; https://epoch.ai/benchmarks?view=graph&tab=eci.

Eriksson, Maria, Erasmo Purificato, Arman Noroozian, et al. 2025. *Can We Trust AI Benchmarks? An Interdisciplinary Review of Current Issues in AI Evaluation*. https://arxiv.org/abs/2502.06559.

Fu, Yujuan, Ozlem Uzuner, Meliha Yetisgen, and Fei Xia. 2025. "Does Data Contamination Detection Work (Well) for LLMs? A Survey and Evaluation on Detection Assumptions." In *Findings of the Association for Computational Linguistics: NAACL 2025*, edited by Luis Chiruzzo, Alan Ritter, and Lu Wang. Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.findings-naacl.291.

Ggaliwango, Marvin, Hellen Nakayiza, Daudi Jjingo, and Joyce Nakatumba-Nabende. 2024. "Prompt Engineering in Large Language Models." In *Data Intelligence and Cognitive Informatics*. https://doi.org/10.1007/978-981-99-7962-2_30.

Google DeepMind. 2026. *Gemini 3.5 Flash*. https://deepmind.google/models/model-cards/gemini-3-5-flash/.

Guha, Neel, Andy K. Zhang, Christine Tsang, Christopher D. Manning, Julian Nyarko, and Daniel E. Ho. 2026. "There is no free benchmark: An institutional view of legal AI benchmarking." *Proceedings of the National Academy of Sciences of the United States of America*, no. 30. https://doi.org/10.1073/pnas.2509757122.

Gundlach, Hans, Jayson Lynch, Matthias Mertens, and Neil Thompson. 2026. *The Price of Progress: Price Performance and the Future of AI*. https://arxiv.org/abs/2511.23455.

Hodak, Miro, David Ellison, Chris Van Buren, Xiaotong Jiang, and Ajay Dholakia. 2024. "Benchmarking Large Language Models: Opportunities and Challenges." In *Performance*

Evaluation and Benchmarking, edited by Raghunath Nambiar and Meikel Poess, vol. 14247. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-031-68031-1_6.

Hutchinson, Ben, Negar Rostamzadeh, Christina Greer, Katherine Heller, and Vinodkumar Prabhakaran. 2022. "Evaluation Gaps in Machine Learning Practice." *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency* (New York, NY, USA), FAccT '22, 1859–76. https://doi.org/10.1145/3531146.3533233.

Jimenez, Carlos E., John Yang, Alexander Wettig, et al. 2024. *SWE-Bench: Can Language Models Resolve Real-World GitHub Issues?* https://arxiv.org/abs/2310.06770.

Kapoor, Sayash, Benedikt Stroebl, Peter Kirgis, et al. 2025. *Holistic Agent Leaderboard: The Missing Infrastructure for AI Agent Evaluation*. https://arxiv.org/abs/2510.11977.

Khairnar, Sushil. 2025. "A Survey of Temporal Drift in Large Language Models." *TechRxiv* 2025 (1030). https://doi.org/10.36227/techrxiv.176184838.81737563/v1.

Koch, Bernard, Emily Denton, Alex Hanna, and Jacob G. Foster. 2021. "Reduced, Reused and Recycled: The Life of a Dataset in Machine Learning Research." *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*. https://arxiv.org/abs/2112.01716.

Lee, Seunghyun, and David Brumley. 2026. *ExploitBench: A Capability Ladder Benchmark for LLM Cybersecurity Agents*. May. https://arxiv.org/abs/2605.14153.

Li, Jeremy, and Andrew Ho. 2026. "GeneBench: Assessing AI Agents for Multi-Stage Inference Problems in Genomics and Quantitative Biology." *bioRxiv*, ahead of print. https://doi.org/10.64898/2026.04.22.720113.

Li, Yucheng, Yunhao Guo, Frank Guerin, and Chenghua Lin. 2024. "An Open-Source Data Contamination Report for Large Language Models." In *Findings of the Association for Computational Linguistics: EMNLP 2024*, edited by Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen. Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-emnlp.30.

Liang, Percy, Rishi Bommasani, Tony Lee, et al. 2023. "Holistic Evaluation of Language Models." *Transactions on Machine Learning Research*. https://arxiv.org/abs/2211.09110.

Mitchell, Margaret, Simone Wu, Andrew Zaldivar, et al. 2019. "Model Cards for Model Reporting." (New York, NY, USA). https://doi.org/10.1145/3287560.3287596.

Mondorf, Philipp, and Barbara Plank. 2024. "Liar, Liar, Logical Mire: A Benchmark for Suppositional Reasoning in Large Language Models." In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, edited by Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen. Association for Computational Linguistics. https://aclanthology.org/2024.emnlp-main.404.

Narayanan, Arvind. 2026. *What Will Be Left for Us to Work on?* Princeton University. https://www.cs.princeton.edu/~arvindn/talks/icml-2026-annotated-slides/#s28.

OpenAI. 2026a. *GPT-5.5 System Card*. https://deploymentsafety.openai.com/gpt-5-5.

OpenAI. 2026b. *Openai-Python: The Official Python Library for the OpenAI API*. Version 2.36.0. https://pypi.org/project/openai/.

Patwardhan, Aditya, Vivek Vaidya, and Ashish Kundu. 2025. *Automated Consistency Analysis of LLMs*. https://arxiv.org/abs/2502.07036.

Pinhanez, Claudio, Paulo Cavalin, Cassia Sanctos, Marcelo Grave, and Yago Primerano. 2025. *The Non-Determinism of Small LLMs: Evidence of Low Answer Consistency in Repetition Trials of Standard Multiple-Choice Benchmarks*. https://arxiv.org/abs/2509.09705.

Polo, Felipe Maia, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. 2024. "tinyBenchmarks: evaluating LLMs with fewer examples." In *Proceedings of the 41st International Conference on Machine Learning*, edited by Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, et al., vol. 235. Proceedings of Machine Learning Research. PMLR. https://proceedings.mlr.press/v235/maia-polo24a.html.

Rabanser, Stephan, Sayash Kapoor, Peter Kirgis, Kangheng Liu, Saiteja Utpala, and Arvind Narayanan. 2026. *Towards a Science of AI Agent Reliability*. https://arxiv.org/abs/2602.16666.

Raji, Inioluwa Deborah, Emily M. Bender, Amandalynne Paullada, Emily Denton, and Alex Hanna. 2021. *AI and the Everything in the Whole Wide World Benchmark*. https://arxiv.org/abs/2111.15366.

Reuel, Anka, Amelia Hardy, Chandler Smith, Max Lamparth, Malcolm Hardy, and Mykel J. Kochenderfer. 2024. *BetterBench: Assessing AI Benchmarks, Uncovering Issues, and Establishing Best Practices*. https://arxiv.org/abs/2411.12990.

Russakovsky, Olga, Jia Deng, Hao Su, et al. 2015. "ImageNet Large Scale Visual Recognition Challenge." *International Journal of Computer Vision*, 211–52. https://doi.org/10.1007/s11263-015-0816-y.

Sahoo, Pranab, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman

Chadha. 2025. *A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications*. https://arxiv.org/abs/2402.07927.

Smullyan, Raymond M. 1981. “What Is the Name of This Book? The Riddle of Dracula and Other Logical Puzzles.” *Critica* 13 (38): 126–30.

Wan, Alexander, Kevin Klyman, Sayash Kapoor, et al. 2025. *The 2025 Foundation Model Transparency Index*. Stanford Center for Research on Foundation Models (CRFM); https://crfm.stanford.edu/fmti/December-2025/paper.pdf.

Wang, Zhun, Nico Schiller, Hongwei Li, et al. 2026. *ExploitGym: Can AI Agents Turn Security Vulnerabilities into Real Attacks?* https://arxiv.org/abs/2605.11086.

Wu, Xuansheng, Wenlin Yao, Jianshu Chen, et al. 2024. “From Language Modeling to Instruction Following: Understanding the Behavior Shift in LLMs After Instruction Tuning.” In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, edited by Kevin Duh, Helena Gomez, and Steven Bethard. Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.naacl-long.130.

Xu, Cheng, Shuhao Guan, Derek Greene, and M-Tahar Kechadi. 2024. *Benchmark Data Contamination of Large Language Models: A Survey*. https://arxiv.org/abs/2406.04244.

Zhou, Jeffrey, Tianjian Lu, Swaroop Mishra, et al. 2023. *Instruction-Following Evaluation for Large Language Models*. https://arxiv.org/abs/2311.07911.

Zhu, Yuxuan, Antony Kellermann, Dylan Bowman, et al. 2025. *CVE-Bench: A Benchmark for AI Agents’ Ability to Exploit Real-World Web Application Vulnerabilities*. https://arxiv.org/abs/2503.17332.